

TECHNICAL ENCRYPTION DISCLOSURE

Vault App Encryption Documentation

Prepared for App Store Connect encryption review

Field	Value
Developer / app owner	Bad Bundle Limited
App	Vault
Bundle identifiers	com.badbundle.vault; com.badbundle.vault.VaultAutofill; com.badbundle.vault.Widgets
Marketing version reviewed	2.0
Source reviewed	Commit 8b559510864bc9a223a17da009edb8b70fd50ed7 on main
Repository	https://github.com/bradleymackey/vault-ios
Distribution assumption	Worldwide App Store distribution, including France
Document date	5 June 2026

Purpose and limitation

This document is a technical description of the encryption and cryptographic functions implemented in Vault. It is intended to support App Store Connect review. It is not legal advice and does not replace any country-specific declaration, government filing, or Apple-provided export-compliance code if Apple requires one.

Executive Summary

Vault is an open source, local-first iOS application for storing secure notes and one-time passcode data. The app contains and implements standard cryptographic algorithms for optional per-item encryption, encrypted vault backups, encrypted transfer/export payloads, one-way local passphrase digests, and HOTP/TOTP code generation.

The app does not implement proprietary or unpublished cryptographic algorithms. The app does use standard encryption algorithms outside Apple's operating-system encryption facilities, principally through CryptoSwift, which the Vault Swift package pins at version 1.9.0. It also uses Apple platform security services, including CryptoKit and the Keychain, for selected local cryptographic and secret-storage functions.

App Store Connect Encryption Responses

Question	Answer	Basis
Does the app use encryption?	Yes	Vault implements encryption and cryptographic functions for user-controlled data protection.
Does the app use standard encryption outside Apple OS APIs?	Yes	AES-GCM and KDF/HMAC functions are implemented through CryptoSwift and app code, in addition to Apple platform security services.
Does the app use proprietary or non-standard cryptography?	No	The disclosed algorithms are published, industry-standard algorithms. Vault does not define a proprietary cipher or unpublished cryptographic algorithm.
Is encryption limited to Apple's operating system?	No	Vault uses Apple Keychain/CryptoKit for some local functions, but AES-GCM and several KDF/HMAC operations are provided outside the Apple OS encryption facilities.

Cryptographic Functions Implemented

Use	Algorithms / API	Implementation details
Encrypted secure notes	AES-GCM via CryptoSwift	User-selected per-item password; 256-bit derived key; random 32-byte IV stored with ciphertext; authentication tag stored separately.
Encrypted backup PDF and QR payloads	AES-GCM via CryptoSwift	Whole-vault encrypted export payload; random padding before encryption; 256-bit key from backup KDF; 32-byte IV and authentication tag stored in the encrypted vault container.
Device transfer/export payloads	AES-GCM via CryptoSwift	Uses the same encrypted vault export path as backup export for local transfer and encoded payloads.
Release backup KDF	PBKDF2-HMAC-SHA384 -> HKDF-HMAC-SHA3-512 -> scrypt	Signature vault.keygen.backup.secure.v1. PBKDF2 iterations 5,452,351; HKDF key length 256 bits; scrypt N=2 ¹⁸ , r=8, p=1; random 48-byte salt.

Cryptographic Functions Implemented (continued)

Use	Algorithms / API	Implementation details
Release item KDF	script -> PBKDF2-HMAC-SHA384	Signature vault.keygen.item.secure.v1. script N=2 ⁸ , r=4, p=1; PBKDF2 iterations 372,002; random 48-byte salt; output key length 256 bits.
Killphrase digests	HMAC-SHA256 via CryptoKit	One-way digest HMAC-SHA256(K, salt phrase); 16-byte per-item salt; 256-bit Keychain-stored HMAC key.
Search passphrase digests	HMAC-SHA256 via CryptoKit	One-way digest HMAC-SHA256(K, salt fold(phrase)); 16-byte per-item salt; 256-bit Keychain-stored HMAC key.
One-time passcodes	HOTP/TOTP using HMAC-SHA1, HMAC-SHA256, or HMAC-SHA512	OTP code generation and validation for stored authenticator secrets; HOTP counter mode and TOTP time-step mode.
Local secret storage	Apple Keychain via SwiftSecurity	Backup password key material can use Keychain user-presence protection. Phrase HMAC keys use Keychain whenUnlocked access so local matching can run silently after device unlock.

Detailed Implementation Notes

Encrypted secure notes

Secure notes may be encrypted by the user with a password. The password is not stored. When the user provides a password, Vault derives a 256-bit key and encrypts the note's encoded contents with AES-GCM. The encrypted item stores the encrypted data, authentication tag, random IV, KDF salt, and KDF signature so the key can be re-derived later from the user's password.

Encrypted backups and transfer payloads

Vault backups are exported as PDF documents containing an encrypted vault payload, including QR-encoded data blocks. The backup payload is encoded, padded with random obfuscation data, and encrypted with AES-GCM. The encrypted vault container stores the ciphertext, authentication tag, IV, KDF salt, and KDF signature. Device transfer/export uses the same encrypted vault payload path.

Release key derivation

Optimized release builds use the secure key derivation profiles. Backup encryption uses vault.keygen.backup.secure.v1: PBKDF2-HMAC-SHA384 with 5,452,351 iterations, then HKDF-HMAC-SHA3-512, then script with N=2¹⁸, r=8, p=1. Item encryption uses vault.keygen.item.secure.v1: script with N=2⁸, r=4, p=1, then PBKDF2-HMAC-SHA384 with 372,002 iterations. Both profiles derive 256-bit keys and use randomly generated 48-byte salts.

Local one-way phrase digests

Vault stores killphrases and search passphrases as one-way HMAC-SHA256 digests rather than plaintext. Each digest uses a 16-byte per-item salt. The HMAC keys are 256-bit random values stored locally in the Keychain. Search passphrases are Unicode-normalized and case-folded before HMAC computation so matching preserves the app's case-insensitive search behavior.

OTP generation

Vault supports HOTP and TOTP one-time passcodes using HMAC-SHA1, HMAC-SHA256, or HMAC-SHA512 according to the stored OTP configuration. These functions generate authentication codes locally from user-provided shared secrets and do not provide encrypted communication or remote data transport.

Apple platform security usage

Vault also relies on iOS file protection, CryptoKit, and Keychain services. SwiftSecurity wraps Keychain access for locally stored secret material. Some Keychain entries use user-presence protection; phrase HMAC keys use whenUnlocked access so local phrase matching can occur after the device is unlocked without prompting and without storing plaintext phrases.

Third-Party and Platform Cryptographic Components

Component	Use
CryptoSwift (Vault package pin: 1.9.0)	Provides AES-GCM, PBKDF2, HKDF, scrypt, and HMAC functionality used by Vault's CryptoEngine and OTP implementation.
CryptoKit	Used for HMAC-SHA256 phrase digest generation and validation with constant-time authentication-code checks.
SwiftSecurity 2.5.0	Used as a Swift wrapper around Apple's Keychain services for local secret material storage.
Apple OS security	Used for Keychain and underlying iOS device/file protection. Vault's disclosed AES-GCM implementation is not limited to Apple OS-provided encryption.

France and Export-Compliance Note

Because this document assumes worldwide App Store distribution, including France, and because Vault implements industry-standard cryptography outside Apple's operating-system encryption facilities, Apple may require a French encryption declaration in App Store Connect. This technical document describes the implementation, but it should not be treated as a substitute for any Apple-requested French declaration form or other country-specific filing.

Info.plist and Build Metadata Note

No app code, public API, schema, or Info.plist change is made by this PDF-only documentation task. Based on the reviewed implementation, a follow-up build metadata change will likely need to set App Uses Non-Exempt Encryption (ITSAppUsesNonExemptEncryption) to YES. If Apple approves submitted export-compliance documentation and provides an App Encryption Export Compliance Code, the build may also need ITSEncryptionExportComplianceCode set to Apple's provided value.

Source Files Reviewed

- Vault/Sources/CryptoEngine/Symmetric/AESGCMEncryptor.swift and AESGCMDecryptor.swift
- Vault/Sources/VaultKeygen/VaultKeyDeriver.swift, VaultKeyDeriverFactory.swift, and VaultKeyDeriver+Keygen.swift
- Vault/Sources/VaultBackup/Encryption and Vault/Sources/VaultBackup/BackupEncoder
- Vault/Sources/VaultFeed/Encryption including item encryption, killphrase digests, and search passphrase digests
- Vault/Sources/CryptoEngine/TOTP/HOTPGenerator.swift and TOTPGenerator.swift
- Vault/Package.swift and Vault/Package.resolved
- VaultApp/VaultApp/Info.plist, VaultApp/VaultAppAutofill/Info.plist, and VaultApp/VaultWidgets/Info.plist

References

- Apple App Store Connect Help, Overview of export compliance: <https://developer.apple.com/help/app-store-connect/manage-app-information/overview-of-export-compliance>
- Apple App Store Connect Help, Export compliance documentation for encryption: <https://developer-mdn.apple.com/help/app-store-connect/reference/app-information/export-compliance-documentation-for-encryption>
- Apple Xcode Help, Add export compliance keys: <https://help.apple.com/xcode/mac/current/en.lproj/dev0dc15d044.html>
- NIST FIPS 197, Advanced Encryption Standard (AES): <https://doi.org/10.6028/NIST.FIPS.197-upd1>
- NIST SP 800-38D, Galois/Counter Mode (GCM) and GMAC: <https://doi.org/10.6028/NIST.SP.800-38D>
- RFC 8018, PKCS #5 / PBKDF2: <https://www.rfc-editor.org/rfc/rfc8018>
- RFC 5869, HKDF: <https://www.rfc-editor.org/rfc/rfc5869>
- RFC 7914, scrypt: <https://www.rfc-editor.org/rfc/rfc7914>
- RFC 2104, HMAC: <https://www.rfc-editor.org/rfc/rfc2104>
- RFC 4226, HOTP: <https://www.rfc-editor.org/rfc/rfc4226>
- RFC 6238, TOTP: <https://www.rfc-editor.org/rfc/rfc6238>
- NIST FIPS 180-4, Secure Hash Standard (SHA-2 family): <https://doi.org/10.6028/NIST.FIPS.180-4>

- NIST FIPS 202, SHA-3 Standard: <https://doi.org/10.6028/NIST.FIPS.202>
- Vault open source repository: <https://github.com/bradleymackey/vault-ios>